🛡️ env-guardian

> The elite, lightning-fast CLI to keep your environment variables perfectly synced.

Ever pushed a broken app to production because you forgot to add a new environment variable to your `.env` file? We've all been there.

env-guardian scans your actual source code, finds every `process.env` variable you are using, and cross-references them with your local `.env` and `.env.example` files. It tells you what's missing, what's empty, and can even fix your template files automatically.

✨ Features

- 🧠 **Smart Scanning:** Reads your codebase in milliseconds to find exactly what environment variables your code actually needs.
- 🔍 **Catch Missing Variables:** Alerts you if your local `.env` is missing variables required by your code.
- 🛡️ **Value Validation:** Detects if you accidentally left an active `.env` variable empty.
- 🔧 **Auto-Fix Magic:** Run with `--fix` and it will automatically append missing variables to your `.env.example` safely (with empty values), so you never leak secrets.
- ⚡ **Blazing Fast:** Built with modern TypeScript, ESM, and asynchronous parallel reading.

📦 Installation

You don't need to install it globally if you don't want to. You can run it directly using your favorite package manager:

```
# Using NPM
npx env-guardian validate

# Using PNPM
pnpm dlx env-guardian validate

# Using Yarn
yarn dlx env-guardian validate
```

Want to add it to your project's CI/CD pipeline? Install it as a dev dependency:

```
npm install -D env-guardian
```

🚀 Usage

`env-guardian` is designed to be ridiculously simple. Just open your terminal in your project folder and run:

1. Validate & Sync (The Command You Want)

This cross-references your code with your `.env` files and tells you what's wrong.

```
npx env-guardian validate
```

Want to automatically update your `.env.example`?
Just add the `--fix` flag. It will safely add any missing keys without touching your existing comments or formatting.

```
npx env-guardian validate --fix
```

2. Just Scan

If you only want to see a list of the variables your project is currently using (without checking the `.env` files), use the scan command:

```
npx env-guardian scan
```

3. Ignore Folders

By default, `env-guardian` ignores `node_modules`, `.git`, and build folders like `dist` or `.next`. If you have a massive monorepo and want to skip a specific folder to make it even faster, use `--ignore`:

```
npx env-guardian validate --ignore "**/legacy-app/**"
```

🤝 Contributing

Found a bug or have an idea to make this better? PRs are always welcome!

1. Clone the repo.
2. Run `npm install`.
3. Make your changes in the `src/` directory.
4. Run `npm run test` to make sure the core regex engine is still happy.
5. Submit your PR!

📄 License

ISC License. See `package.json` for details. Built for developers, by developers.